[승코의 차근차근 Flutter] 04. 기본 위젯

📦 Container 위젯

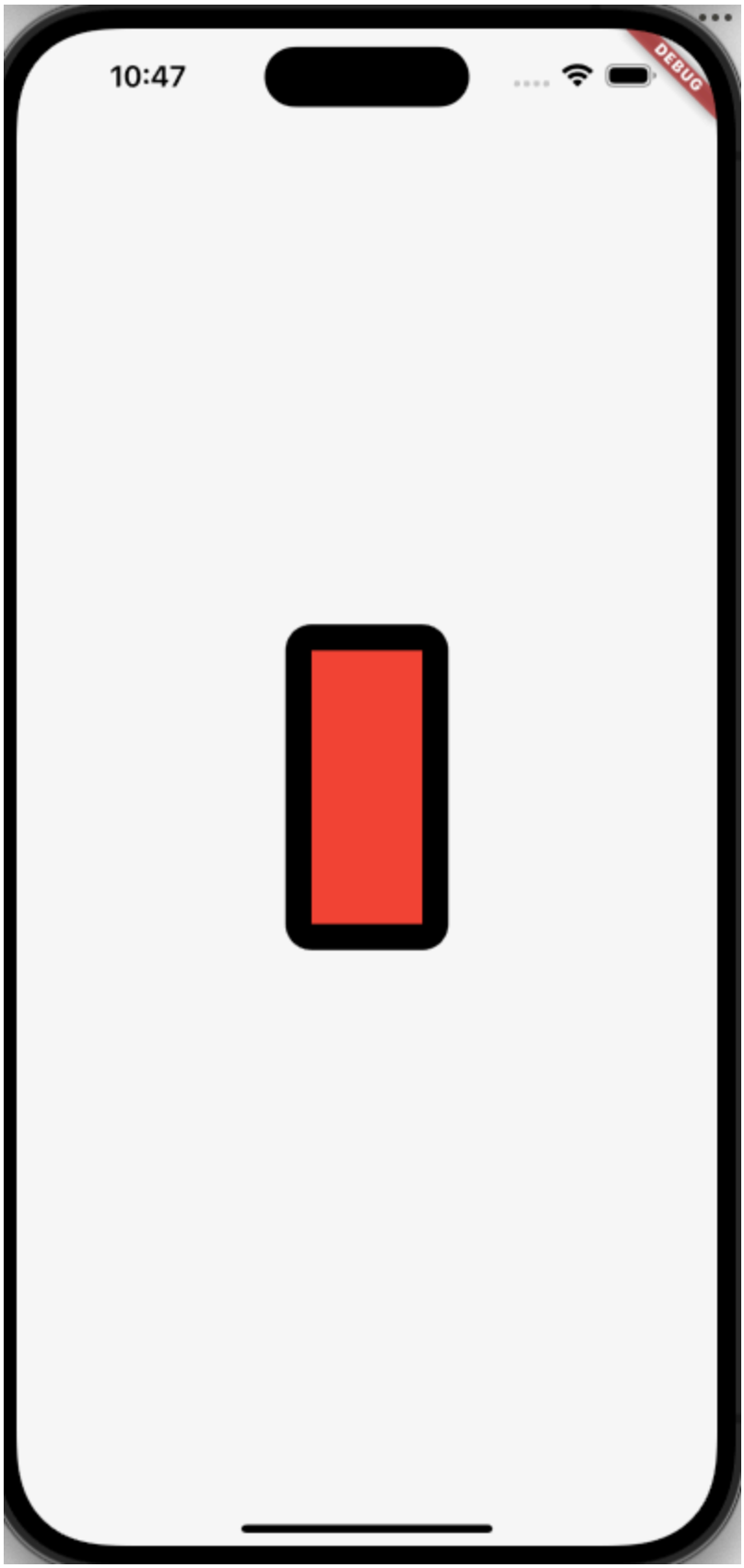
✅ **역할**: 자식 위젯을 감싸고, 크기, 배경색, 테두리, 여백 등을 꾸밀 수 있는 다재다능한 박스 위젯.

🧠 **이해 포인트**:

- `Container`는 UI를 디자인할 때 가장 많이 사용하는 위젯.
- `child`로 다른 위젯을 하나만 포함 가능.
- `decoration` 속성을 통해 배경색, 테두리(border), 둥근 모서리(borderRadius) 등을 설정 가능.

예시 요약:

```
Container(
  width: 100,
  height: 200,
  decoration: BoxDecoration(
    color: Colors.red,
    border: Border.all(color: Colors.black, width: 2),
    borderRadius: BorderRadius.circular(12),
  ),
)
```


10:47
DEBUG

→ 빨간 배경에 검은 테두리, 둥근 모서리를 가진 100x200 박스 생성.

👆 GestureDetector 위젯

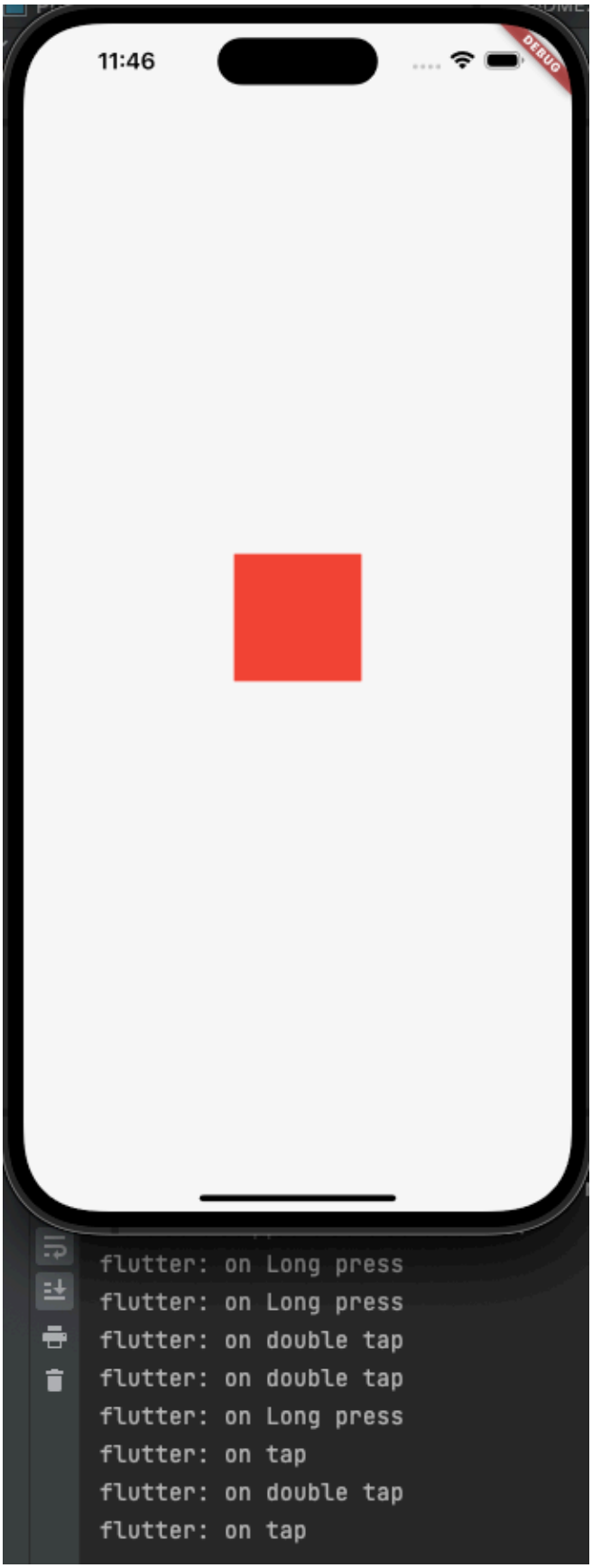
✅ **역할**: 터치(제스처)를 감지하고 동작을 정의하는 위젯.

🧠 **이해 포인트**:

- 단순 클릭부터 드래그, 더블탭, 길게 누르기 등 다양한 제스처 이벤트를 감지.
- 직접 클릭 이벤트를 만들고 싶을 때 유용.

자주 사용하는 콜백들:

- `onTap`: 한 번 탭
- `onDoubleTap`: 두 번 연속 탭
- `onLongPress`: 길게 누르기
- `onPanUpdate`: 드래그 중
- `onScaleUpdate`: 핀치 줌(확대/축소)

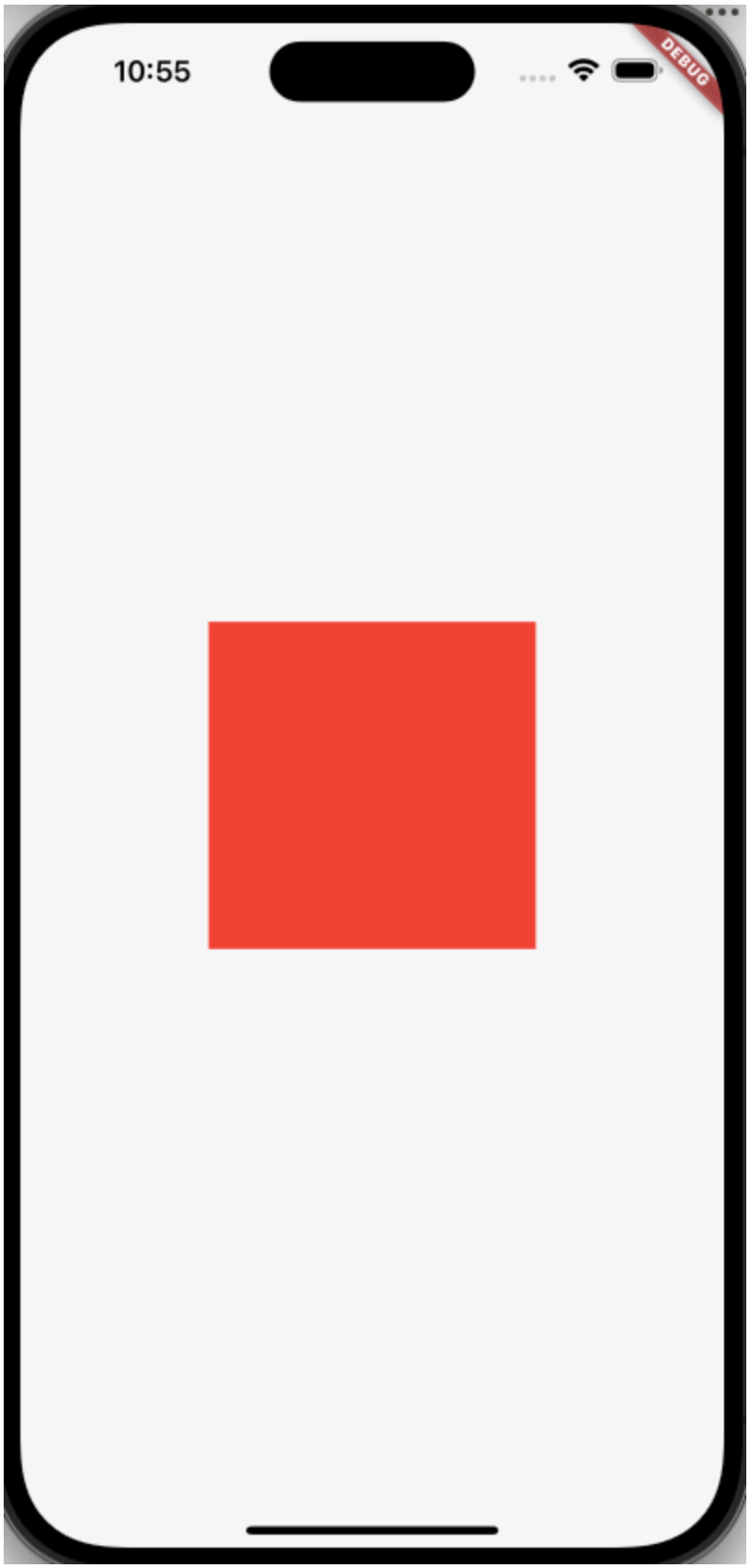

11:46
DEBUG
flutter: on Long press
flutter: on Long press
flutter: on double tap
flutter: on double tap
flutter: on Long press
flutter: on tap
flutter: on double tap
flutter: on tap

📏 SizedBox 위젯

✅ **역할**: 정해진 너비와 높이 공간을 차지하는 "공간 전용 위젯".

🧠 **이해 포인트**:

- 빈 공간을 만들거나 간격을 줄 때 많이 씀.
- `Container`는 배경이나 테두리 등 꾸미는 역할도 가능하지만 `SizedBox`는 오직 크기만 설정.
- `const SizedBox()`를 쓰면 성능적으로 더 가볍다.


10:55
DEBUG

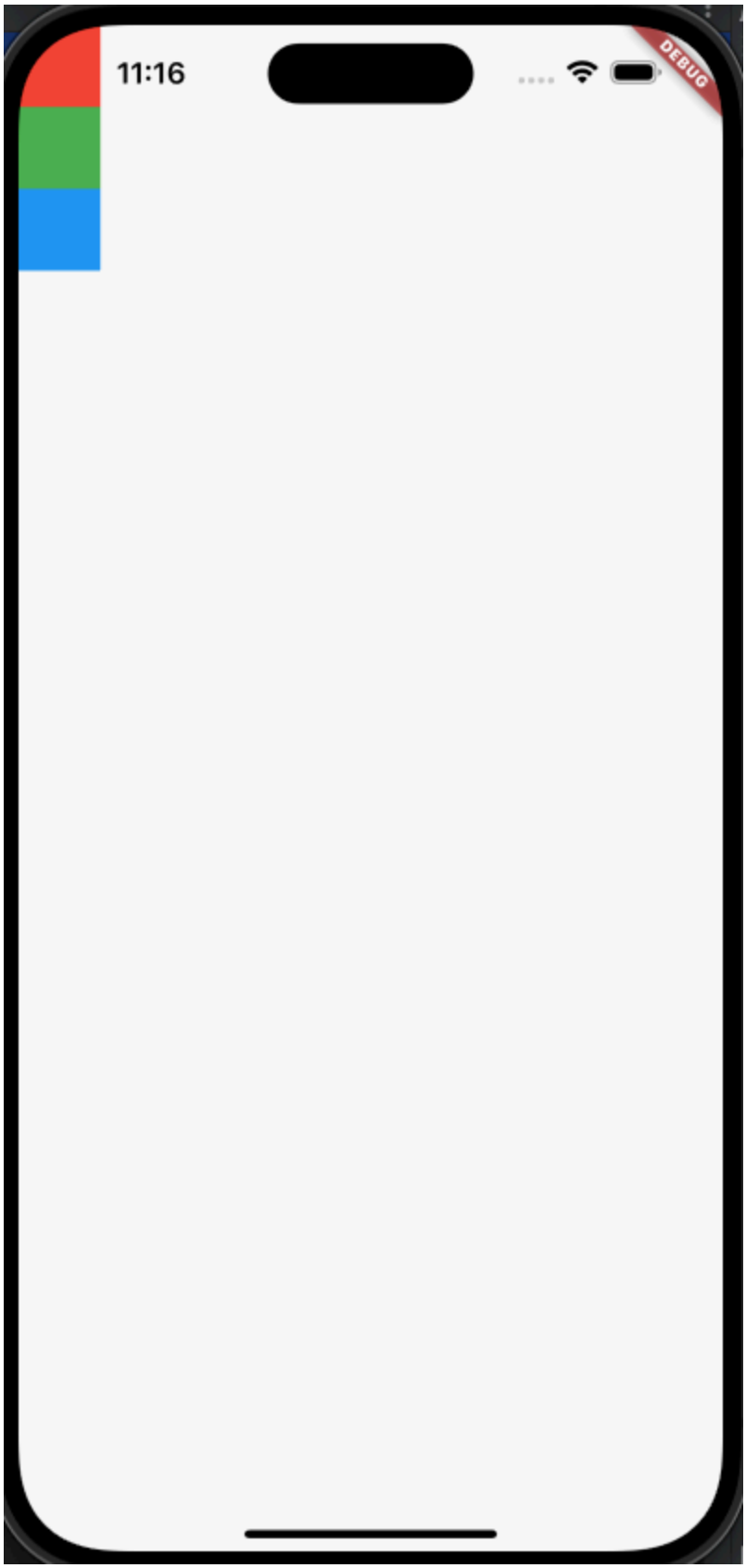
📐 배치 관련 위젯들

📍 Column / Row 위젯

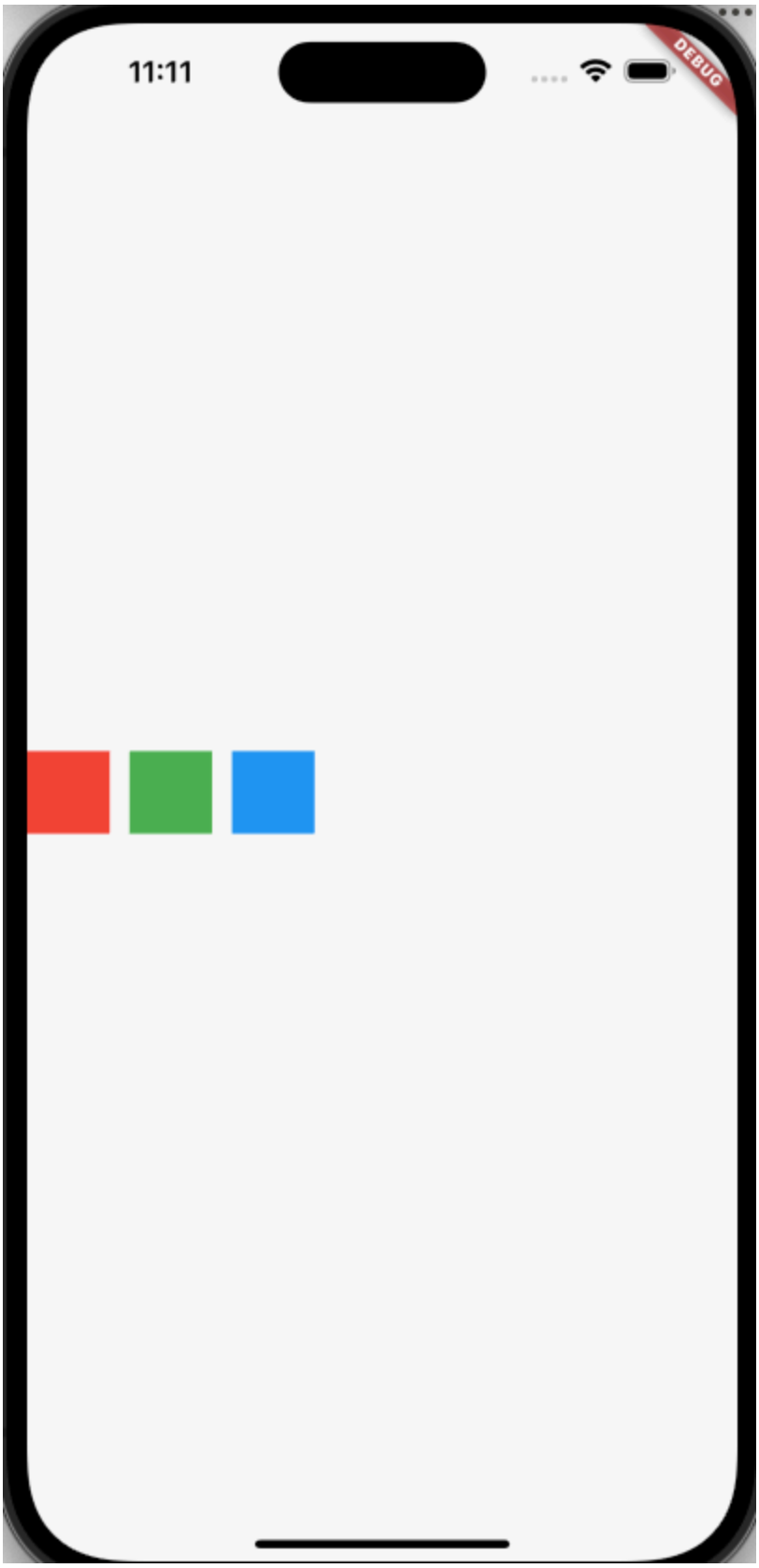
Column: 세로 방향으로 위젯을 쌓음

Row: 가로 방향으로 위젯을 나열

```
Column(
  children: [
    Text('위'),
    Text('아래'),
  ]
)
```


11:16
DEBUG

```
Row(
  children: [
    Icon(Icons.star),
    Text('별'),
  ]
)
```

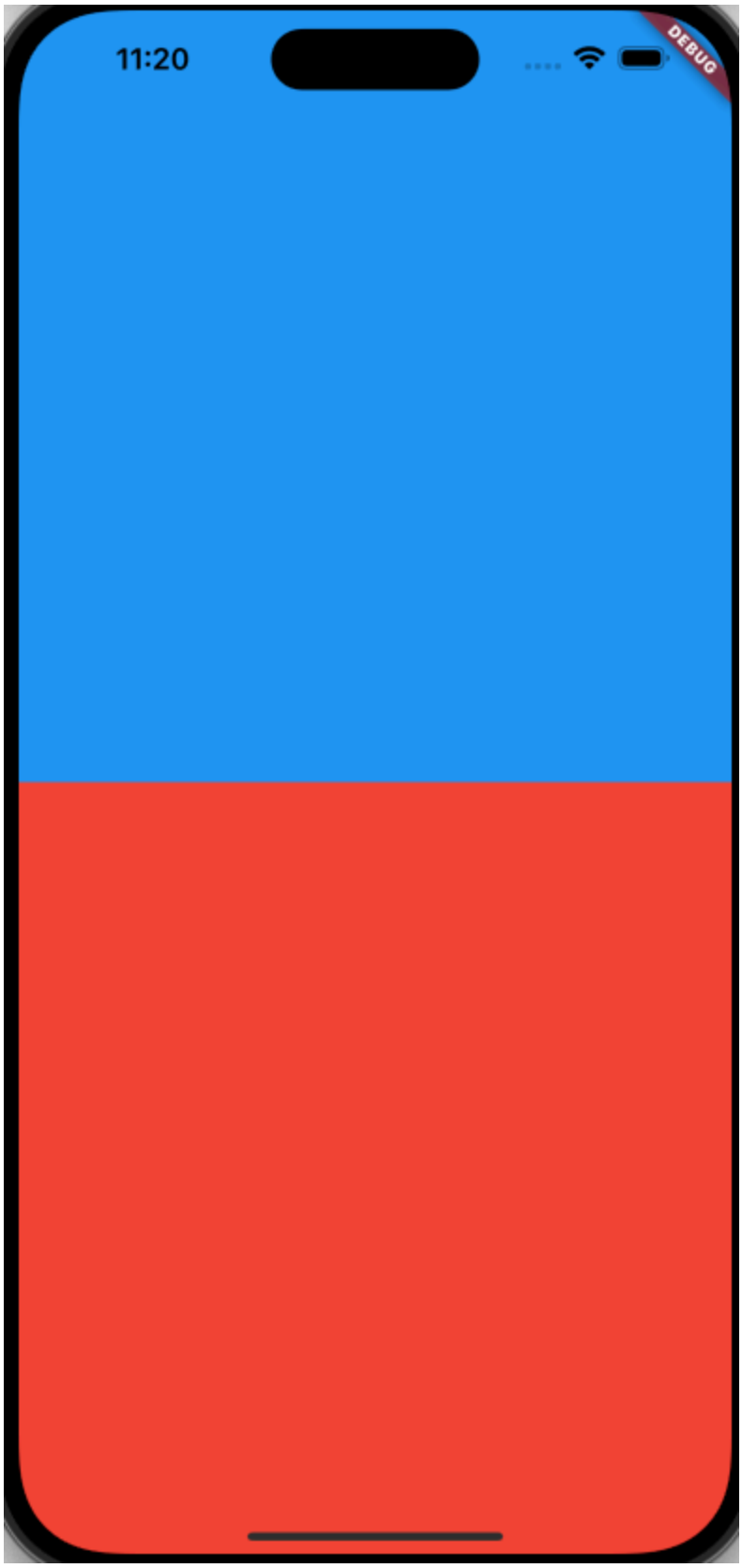

11:11
DEBUG

주요 속성

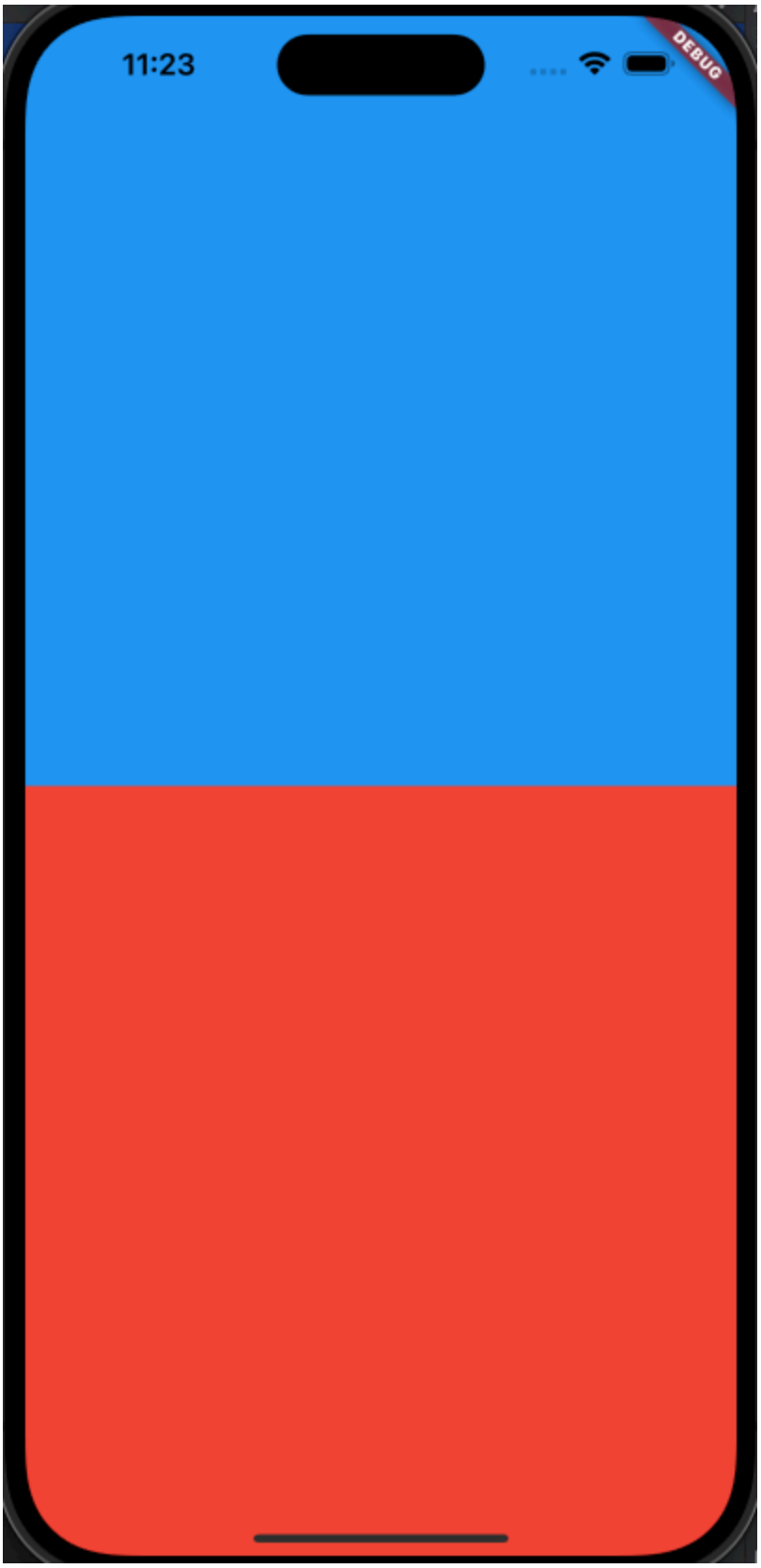
- `mainAxisAlignment`: 주축 정렬 (Column이면 위↕아래 / Row면 왼↔오른)
- `crossAxisAlignment`: 반대축 정렬 (Column이면 왼↔오른 / Row면 위↕아래)

🧮 Flexible / Expanded

- `Flexible`: 주어진 공간을 비율(flex 값)로 나눠 차지함. 공간이 남으면 줄어들 수 있음.


11:20
DEBUG

- `Expanded`: 무조건 가능한 최대한의 공간을 차지함 (Flexible보다 더 확장하려 함)


11:23
DEBUG

```
Column(
  children: [
    Flexible(flex: 1, child: Container(color: Colors.red)),
    Flexible(flex: 2, child: Container(color: Colors.blue)),
  ],
)
```

→ 빨간 박스 1/3, 파란 박스 2/3 차지

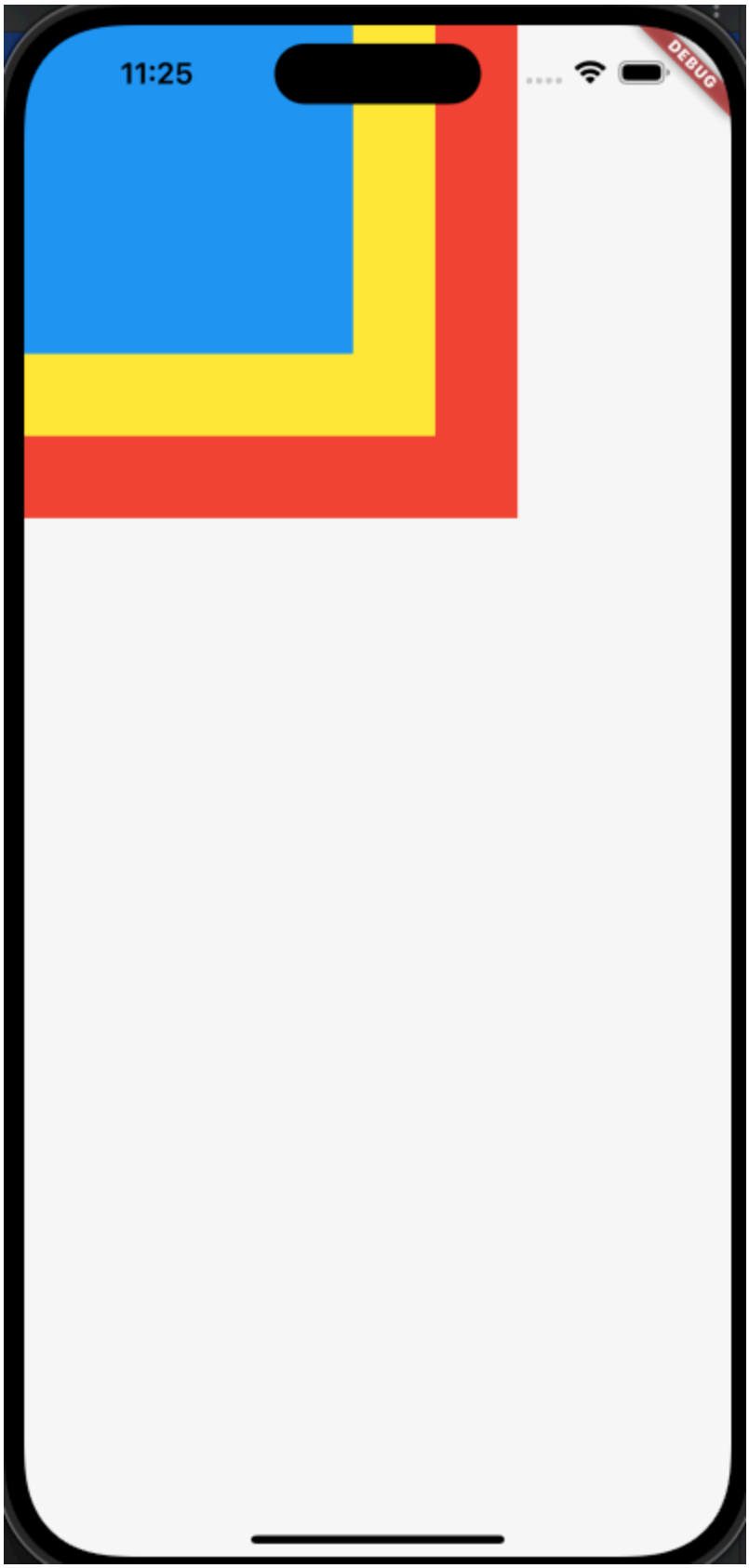
🗂️ Stack 위젯

✅ **역할**: 위젯을 겹쳐서 배치할 수 있는 위젯.

🧠 **이해 포인트**:

- z축(위에 얹는 순서)에 따라 위젯이 겹쳐지며, 가장 마지막 위젯이 가장 위에 있음.
- 좌표를 지정하려면 `Positioned` 위젯 사용 가능.

```
Stack(
  children: [
    Container(width: 300, height: 300, color: Colors.red),
    Container(width: 200, height: 200, color: Colors.yellow),
    Container(width: 100, height: 100, color: Colors.blue),
  ],
)
```

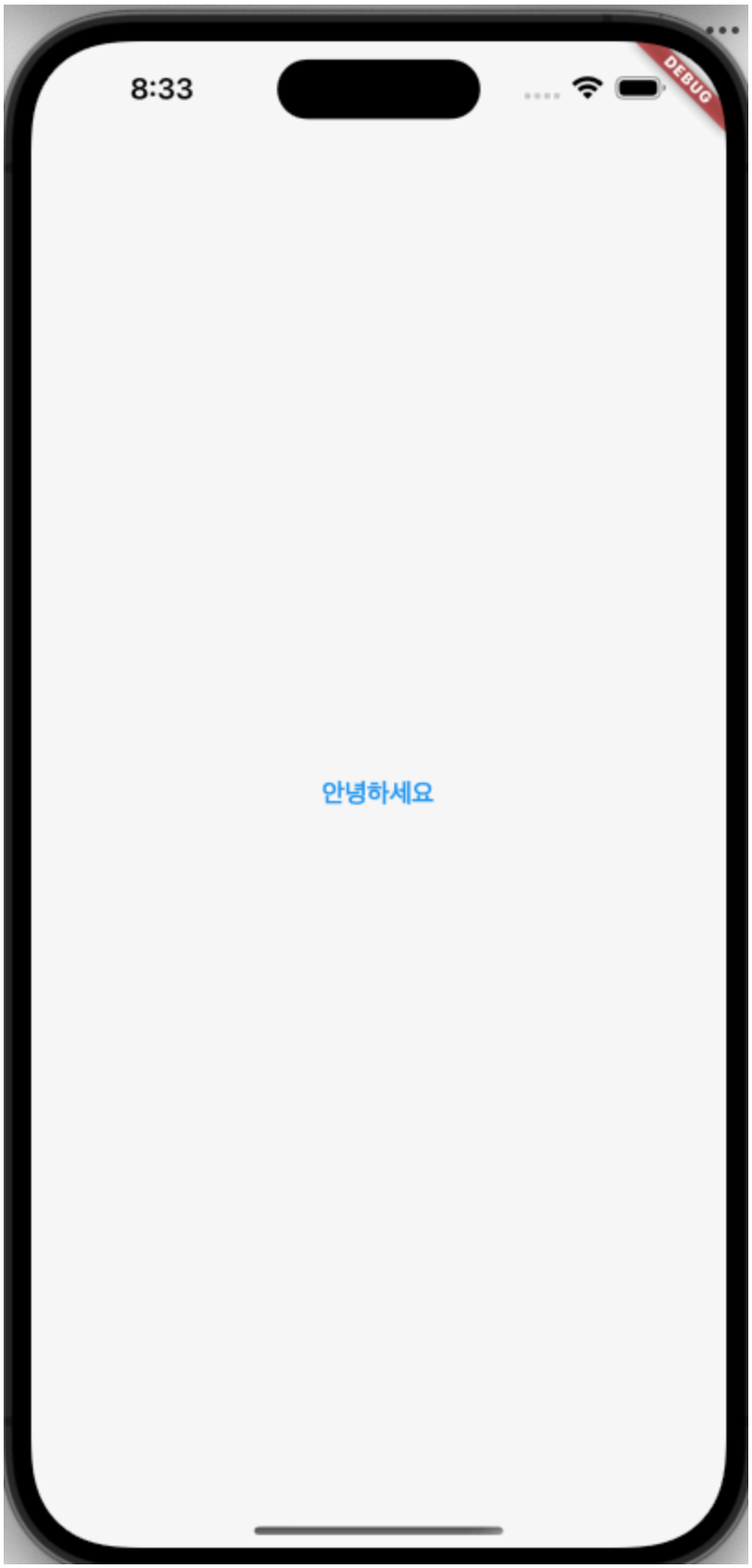

11:25
DEBUG

🔤 텍스트 관련 위젯

📄 Text

- 글자를 출력하는 위젯.
- `style:` 속성으로 글자 색상, 굵기, 크기, 정렬 등을 지정할 수 있음.

```
Text(
  '안녕하세요',
  style: TextStyle(fontSize: 20, color: Colors.blue),
)
```


8:33
DEBUG
안녕하세요

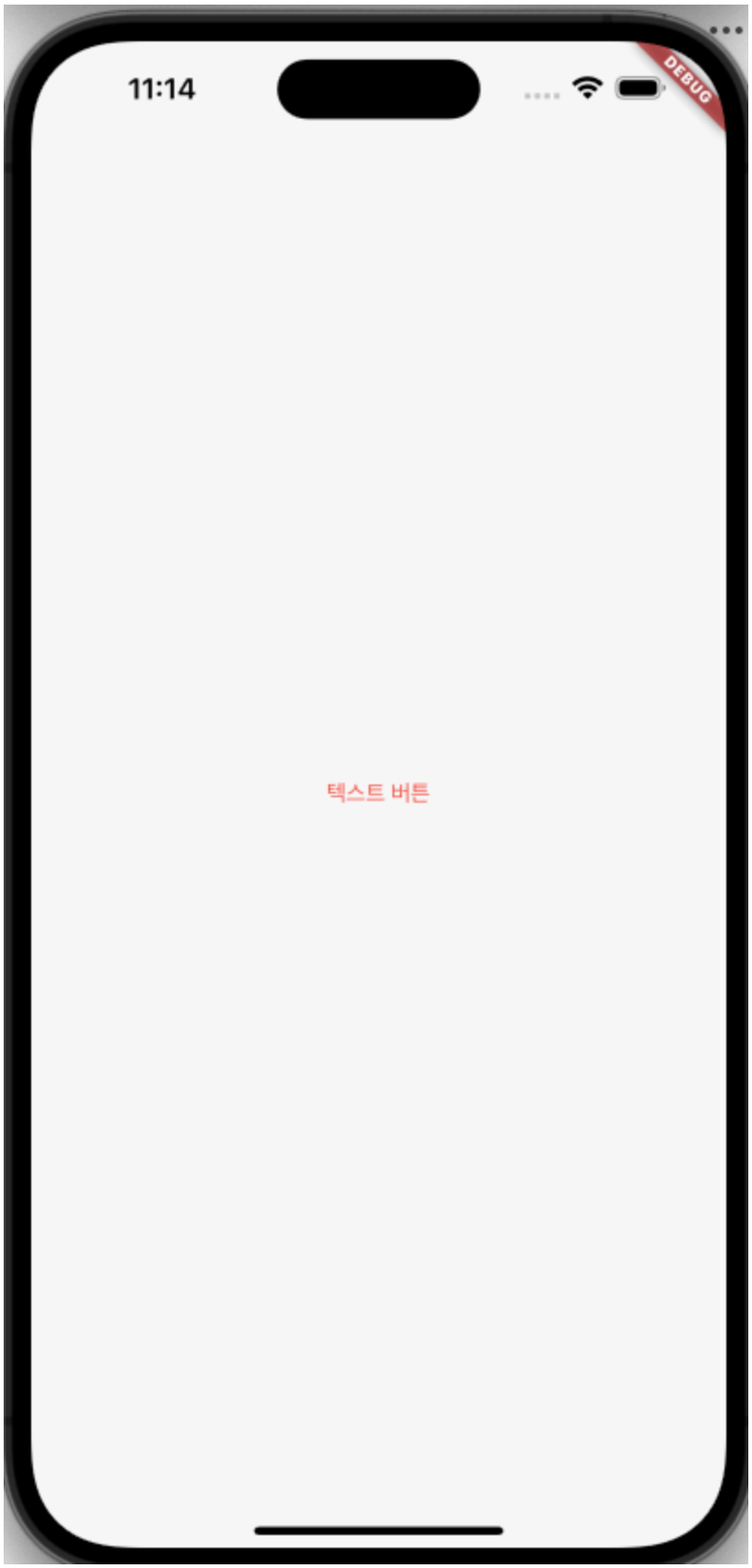
🔘 버튼 위젯 종류

위젯	설명	특징
`TextButton`	기본 글자 버튼	평면 형태, 간단한 UI
`OutlinedButton`	외곽선만 있는 버튼	테두리 강조
`ElevatedButton`	배경색/그림자 버튼	입체감 있고 중요도 높음
`IconButton`	아이콘 클릭	아이콘만 있는 버튼
`FloatingActionButton`	둥근 버튼	화면 하단 플로팅 버튼용

TextButton

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MyApp()
  );
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: TextButton(
            //클릭 시 실행
            onPressed: () {},
            //스타일 지정
            style: TextButton.styleFrom(
              //주 색상 지정
              foregroundColor: Colors.red,
            ),
```

```
          //버튼에 넣을 위젯
          child: Text('텍스트 버튼'),
        ),
      ),
    ),
  );
 }
}
```


11:14
DEBUG
텍스트 버튼

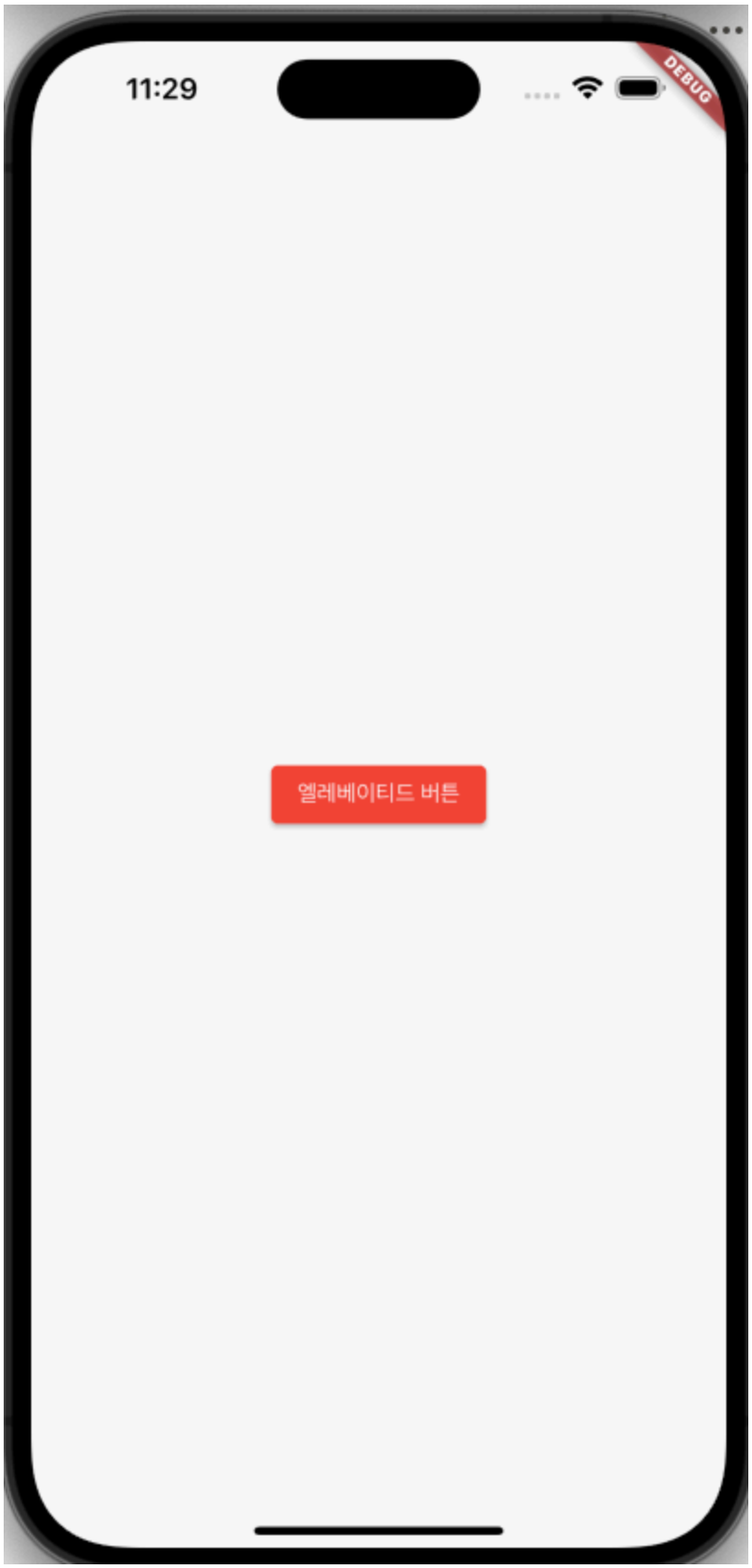
OutlinedButton

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MyApp()
  );
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: OutlinedButton(
            //클릭 시 실행할 함수
            onPressed: () {},
            //버튼 스타일
            style: OutlinedButton.styleFrom(
              foregroundColor: Colors.red,
            ),
            //버튼에 들어갈 위젯
            child: Text('아웃라인드 버튼'),
          )
        ),
      ),
    );
  }
}
```

11:24
DEBUG
아웃라인드 버튼

ElevatedButton

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MyApp()
  );
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: ElevatedButton(
            onPressed: (){},
            style: ElevatedButton.styleFrom(
              backgroundColor: Colors.red,
            ),
            child: Text('엘레베이티드 버튼'),
        ),
      ),
      ),
    );
  }
}
```


11:29
DEBUG
엘레베이티드 버튼

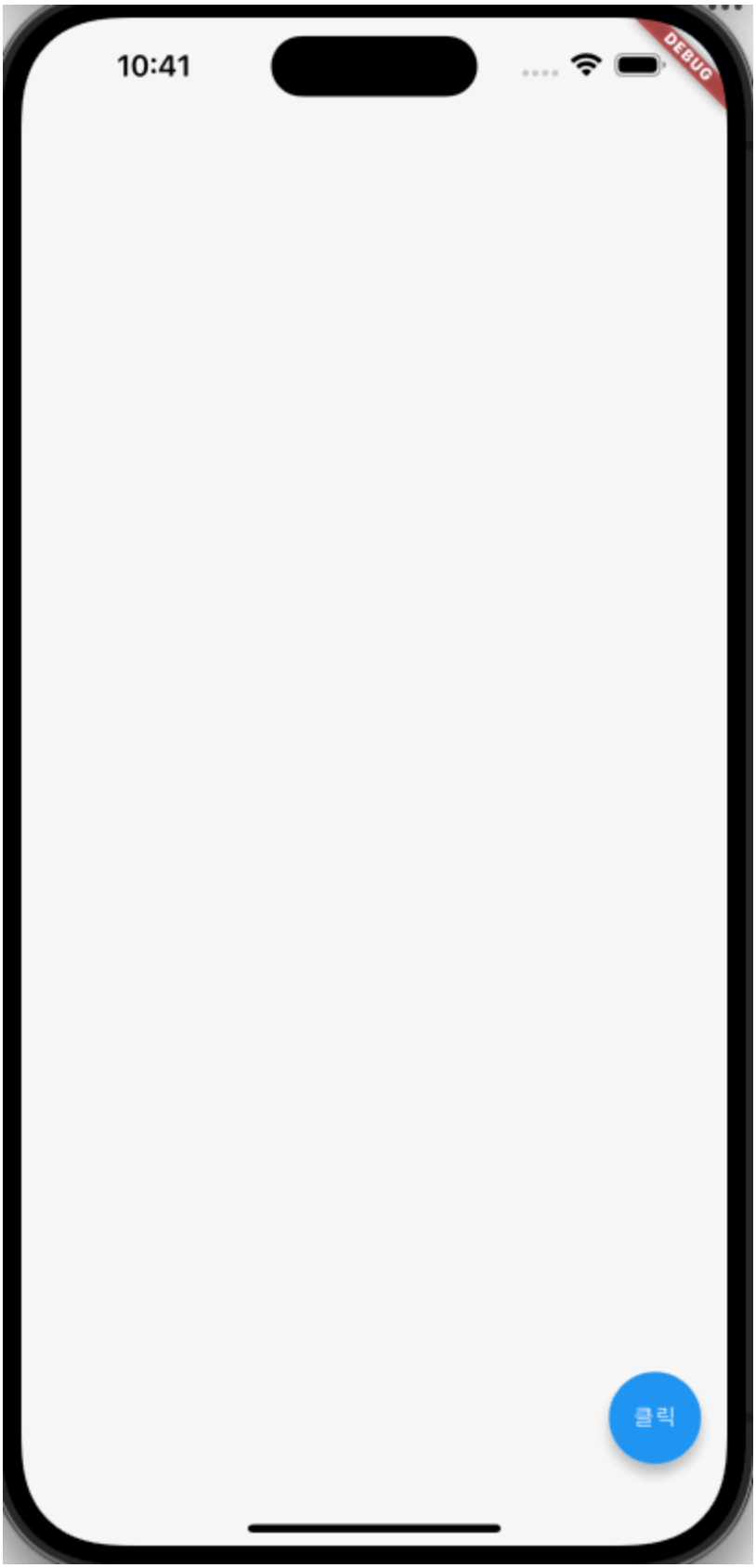
IconButton

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MyApp()
  );
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: IconButton(
            onPressed: () {},
              icon: Icon(
                Icons.home,
              ),
          )
      ),
      ),
    );
  }
}
```

FloatingActionButton

```
import 'package:flutter/material.dart';

  void main() {
    runApp(
```

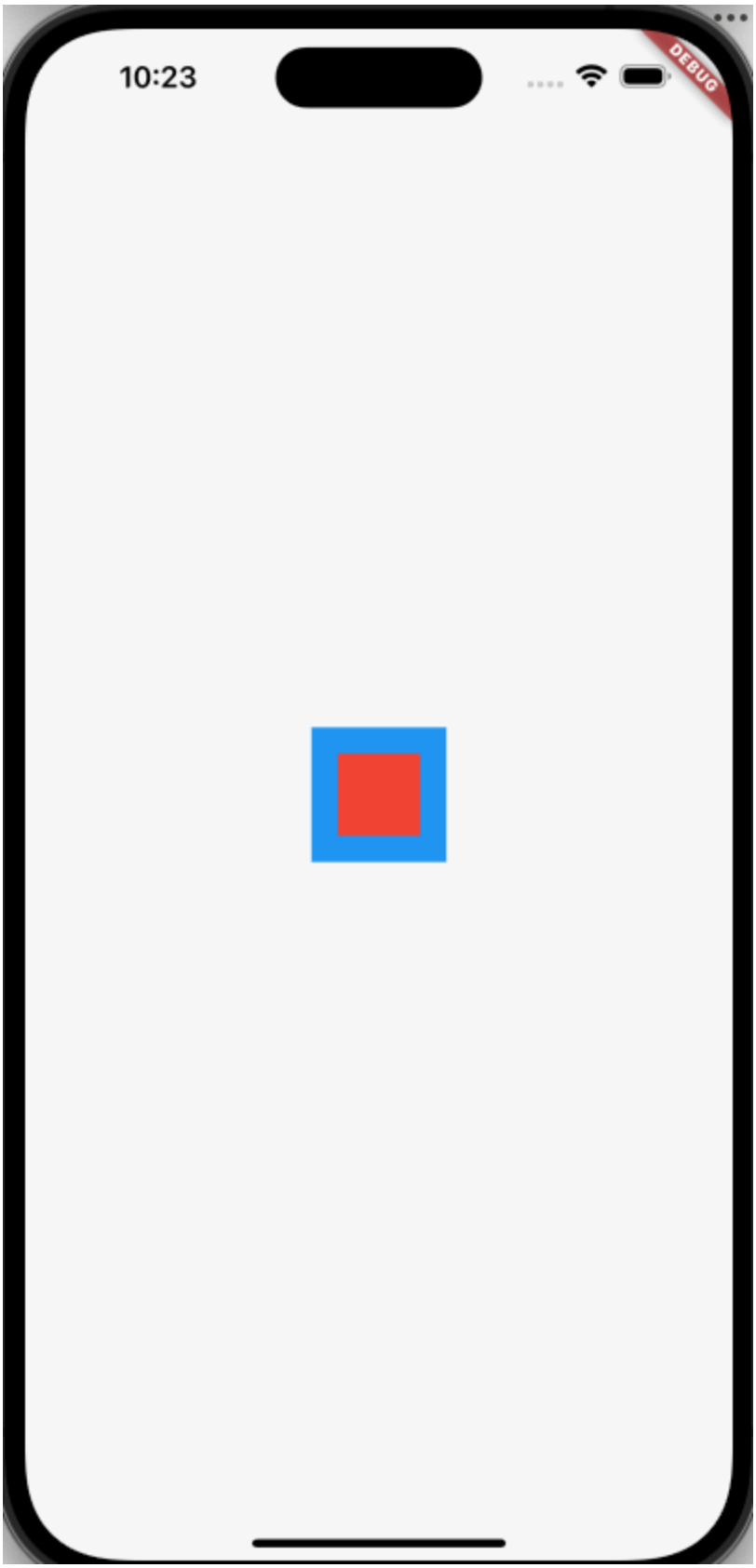
```
    FloatingActionButtonExample()
  );
}

class FloatingActionButtonExample extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        floatingActionButton: FloatingActionButton(
          //클릭했을 때 실행할 함수
          onPressed: () {},
          child: Text('클릭'),
        ),
        body: Container(),
      ),
    );
  }
}
```


10:41
DEBUG
클릭

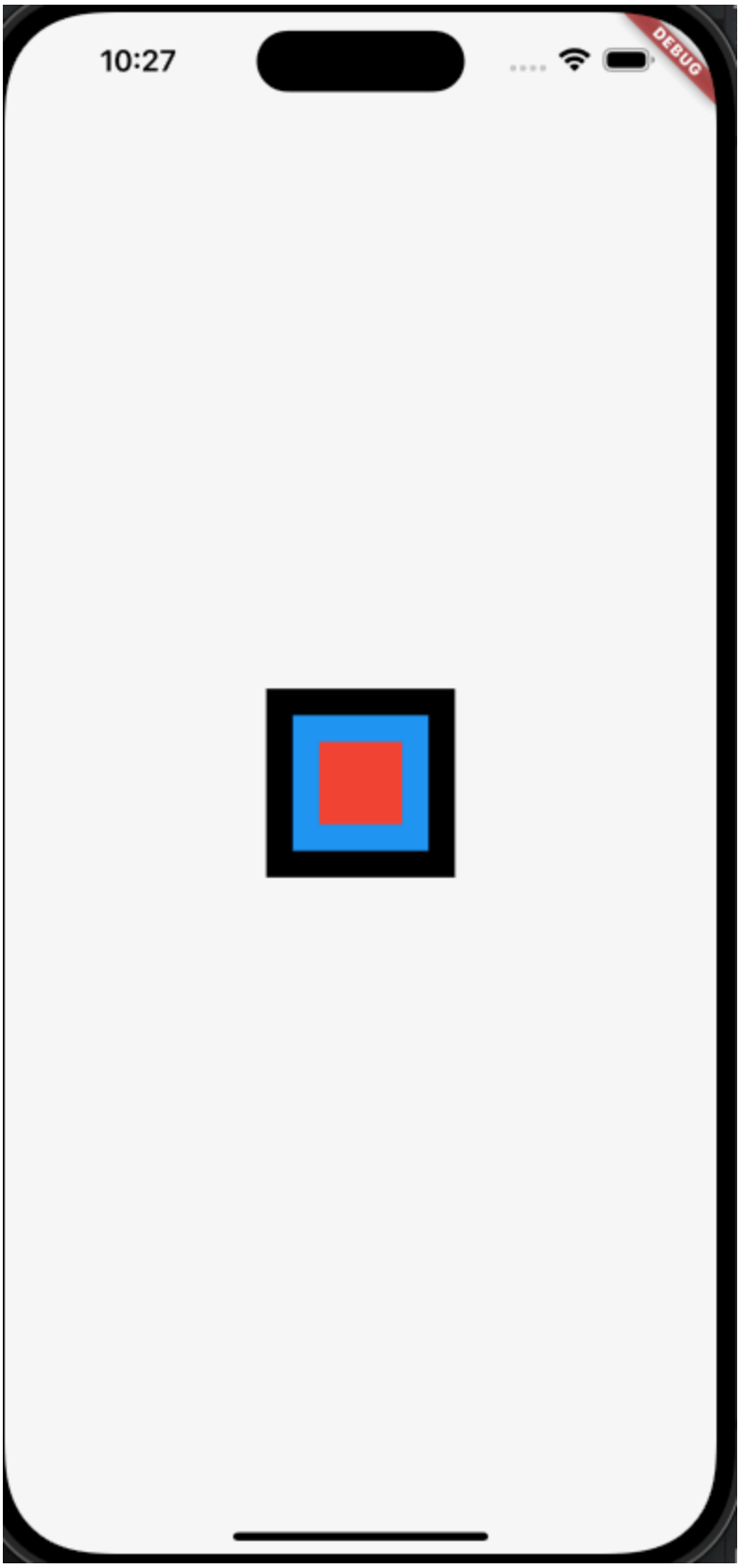
📏 Padding vs Margin

- `Padding`: 안쪽 여백
- `Margin`: 바깥 여백
- 둘 다 `EdgeInsets` 클래스로 설정

```
Padding(
  padding: EdgeInsets.all(16),
  child: Container(color: Colors.red),
)
```


10:23
DEBUG

```
Container(
  margin: EdgeInsets.only(top: 10),
  child: Text('위에 여백만 있음'),
)
```


10:27
DEBUG

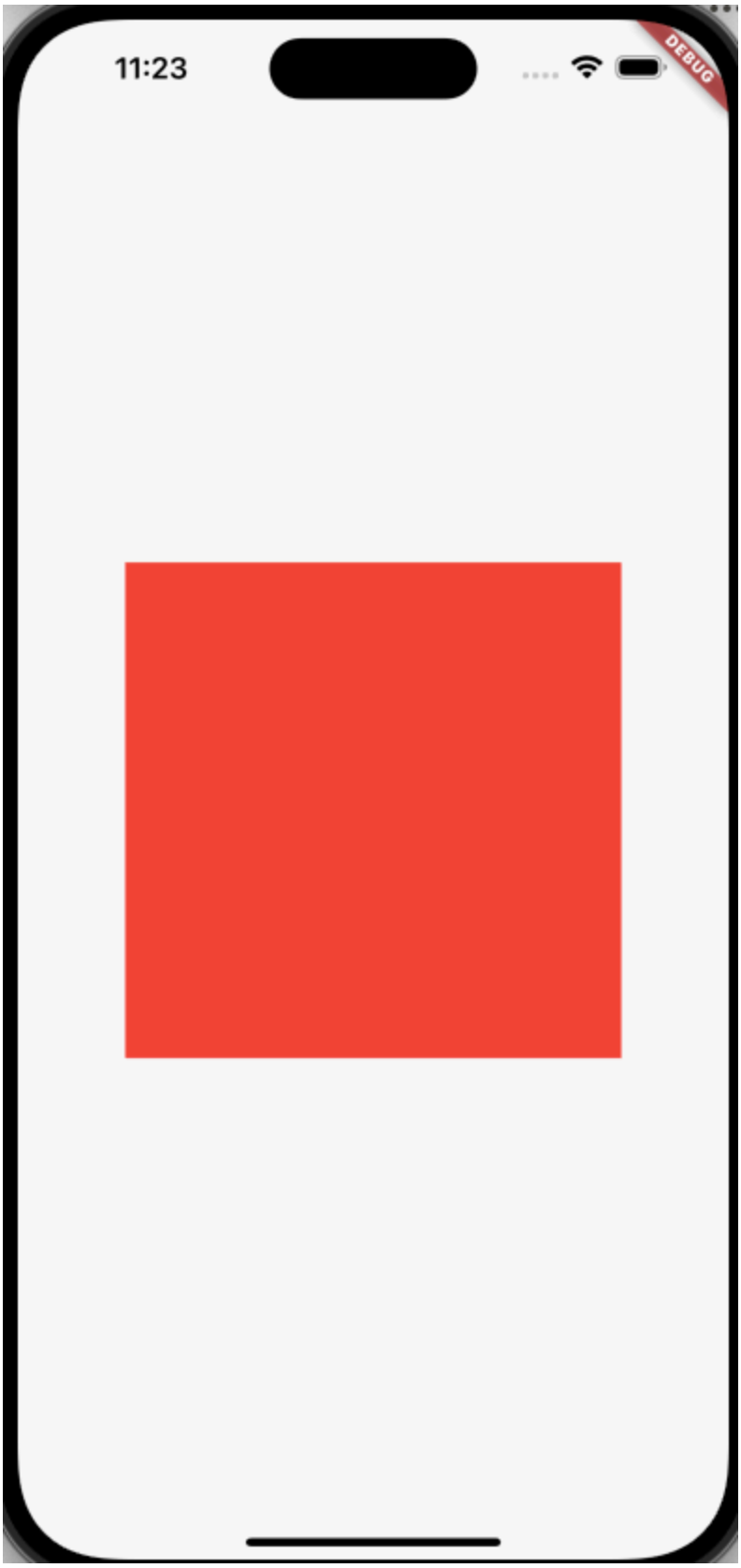
📱 SafeArea 위젯

✅ **역할**: 노치, 상태바 등 스마트폰의 안전 영역을 피해서 레이아웃을 자동으로 조정해주는 위젯.

🧠 **이해 포인트**:

- iOS, Android 기기마다 상단/하단 시스템 UI가 달라서, 레이아웃이 겹칠 수 있는데 이를 방지해줌.

```
SafeArea(
  child: Text('상단 상태바를 피해서 표시됨'),
)
```


11:23
DEBUG

📌 마무리 요약

위젯	역할
`Container`	박스 디자인, 배경색, 테두리 등 꾸밈
`GestureDetector`	터치 제스처 인식
`SizedBox`	단순한 공간 확보용 위젯
`Row`, `Column`	가로/세로 배치
`Flexible`, `Expanded`	공간을 비율 또는 최대한 확장
`Stack`	겹치는 레이아웃 구성
`Text`, `TextStyle`	글자 출력 및 꾸미기
버튼 위젯들	사용자의 입력을 받는 UI
`Padding`, `Margin`	여백 설정
`SafeArea`	화면의 안전한 영역 보장